

Temporal Flight Delay Severity Prediction Using Deep Tabular Models and a Calibrated TabTransformer

Oscar Fabrizio de Alba Gutiérrez
Artificial Intelligence Engineering
Universidad Panamericana
Aguaascalientes, Mexico

Abstract—Flight delay prediction is an important problem in air transportation because delays affect passengers, airlines, airport operations, and downstream scheduling. This paper presents a deep learning pipeline for temporal multiclass flight delay severity prediction using real-world 2025 flight records. The project processes 6,981,533 raw flight records and produces a cleaned model-ready dataset of 6,857,418 flights. The final target is defined as a three-class severity problem: On time, Delay, and Long delay. Unlike random split experiments, the evaluation uses a temporal split, where months 1–9 are used for training, month 10 for validation, and months 11–12 for testing. This design simulates a more realistic deployment setting, where future flights are predicted from past data.

Three PyTorch deep learning models were compared: an MLP with categorical embeddings, a Wide and Deep neural network, and a TabTransformer-style model with categorical and numerical feature tokens. Because the dataset is imbalanced, Macro F1-score was selected as the main evaluation metric instead of accuracy alone. The calibrated TabTransformer achieved the best test Macro F1-score of 0.3905, outperforming the MLP and Wide and Deep models in balanced class performance. The model also improved detection of the Long delay class, which is the most difficult and operationally important class. Finally, a Python graphical inference interface was implemented to load the trained calibrated TabTransformer and estimate calibrated delay severity probabilities for selected flights.

Index Terms—flight delay prediction, deep learning, tabular data, TabTransformer, Wide and Deep learning, class imbalance, Macro F1-score, temporal validation, PyTorch

I. INTRODUCTION AND MOTIVATION

Flight delays are a persistent operational problem in commercial aviation. A delayed flight can affect passenger satisfaction, aircraft utilization, crew scheduling, airport resources, and connecting flights. Even when a delay is not severe, its effect can propagate through later operations. For airlines and airports, being able to estimate delay risk before the flight takes place can support better planning and more informed decisions.

The U.S. Bureau of Transportation Statistics (BTS) commonly considers an arrival delayed when the aircraft arrives 15 minutes or more after the scheduled arrival time [?]. However, a binary delayed/not-delayed classification does not fully describe the severity of the event. A flight delayed by 20 minutes is not equivalent to a flight delayed by more than 90

minutes. The second case is more disruptive and more relevant from an operational perspective.

For this reason, this work models flight delay prediction as a three-class severity classification problem. The final target classes are:

- **On time:** arrival delay less than or equal to 15 minutes.
- **Delay:** arrival delay from 16 to 90 minutes.
- **Long delay:** arrival delay greater than 90 minutes.

The project originally considered a five-class target. However, that definition created severe imbalance, especially for very long delay classes, which represented less than 1% of the dataset. This made the classification problem unstable and caused early models to collapse toward either the majority class or minority classes depending on the imbalance strategy. The final three-class target was selected to preserve operational meaning while making the task more learnable.

The main challenge in this project is that delay severity is naturally imbalanced. Most flights are on time, while long delays are much less frequent. This makes accuracy an insufficient evaluation metric. A model can obtain high accuracy by predicting mostly the majority class and still fail to detect the minority Long delay class. Therefore, this paper uses Macro F1-score as the main metric because it gives equal importance to each class.

The main contributions of this paper are:

- A complete preprocessing pipeline for a large 2025 flight delay dataset.
- A leakage-safe feature set using only information available before the flight.
- A temporal evaluation strategy that avoids overly optimistic random splits.
- A comparison of three deep learning models for tabular delay severity prediction.
- A calibrated TabTransformer model that achieves the best balanced performance.
- A Python graphical interface for estimating calibrated delay probabilities from user-selected flight features.

II. STATE OF THE ART

Flight delay prediction has been studied using statistical learning, machine learning, and deep learning methods. Previous work has shown that aviation delay patterns can be learned

from operational data, but prediction quality depends strongly on the information available at prediction time. Features such as previous aircraft delay, weather, air traffic congestion, and airport operations can significantly improve performance, but they may not always be available before a flight is scheduled to depart.

Machine learning approaches for flight delay prediction often use large aviation datasets and classification or regression models. Big data-driven flight delay studies have shown that machine learning can support delay forecasting and operational planning [2]. However, many highly predictive variables are also close to the actual delay event, which can create data leakage if they are used incorrectly. In this work, leakage prevention is an explicit design constraint: only pre-flight information is used.

Deep learning for tabular data remains a challenging research area. Unlike images or text, tabular datasets contain mixed numerical and categorical variables with heterogeneous meanings. A common strategy is to represent high-cardinality categorical variables with learned embeddings. This is useful in this project because features such as route, origin airport, destination airport, and carrier contain many categories.

Wide and Deep learning was introduced to combine direct linear memorization with deep nonlinear generalization [3]. In this project, the Wide and Deep model is used to combine direct numerical signals with dense categorical embeddings. This allows the model to learn both simple relationships and more complex interactions.

Transformer-based models have also been adapted to tabular data. The Transformer architecture introduced self-attention as a mechanism for learning relationships between tokens [4]. TabTransformer applies this idea to tabular data by transforming categorical embeddings into contextual representations through self-attention [5]. FT-Transformer and related studies have further evaluated Transformer-based architectures for tabular problems and shown that attention-based tabular models can be competitive under appropriate settings [6].

This project builds on these ideas by comparing an embedding-based MLP, a Wide and Deep neural network, and a TabTransformer-style architecture on a temporal, imbalanced, multiclass flight delay severity task.

III. METHODOLOGY

A. Dataset Description

The dataset was obtained from the Airline On-Time Performance Data database provided by the U.S. Bureau of Transportation Statistics (BTS) through the TranStats platform [1]. This database contains scheduled and actual departure and arrival times reported by certified U.S. air carriers. According to BTS, the database is available from 1987 to 2025 with monthly frequency. Although historical records are available since 1987, this work only uses 2025 records because they provide the most recent operational patterns and already contain enough observations for large-scale deep learning experiments. The dataset downloaded contains real-world flight records for 2025. Twelve monthly CSV files were merged into one raw

dataset. The original unified dataset contained 6,981,533 rows and 33 columns. After cleaning, the final full clean dataset contained 6,857,418 rows and 42 columns. The model-ready dataset also contained 6,857,418 rows, but only 24 selected columns were preserved for modeling.

Table I summarizes the dataset versions generated during the pipeline.

TABLE I
DATASET VERSIONS GENERATED DURING PREPROCESSING

Dataset	Rows	Columns
Raw unified dataset	6,981,533	33
Clean full dataset	6,857,418	42
Model-ready dataset	6,857,418	24

Cancelled flights were removed because they do not have a valid completed arrival delay value. Rows with missing arrival delay were also removed. This filtering removed 124,115 rows. Negative arrival delay values were converted to zero because they indicate early arrivals, not negative delay.

The final target class distribution is shown in Table II. The On time class represents 78.78% of the data, while Long delay represents only 4.93%. This confirms that the task remains imbalanced even after redefining the target into three classes.

TABLE II
FINAL THREE-CLASS DELAY SEVERITY DISTRIBUTION

Class	Label	Count	Percentage
0	On time (≤ 15 min)	5,402,270	78.78%
1	Delay (16–90 min)	1,116,997	16.29%
2	Long delay (> 90 min)	338,151	4.93%

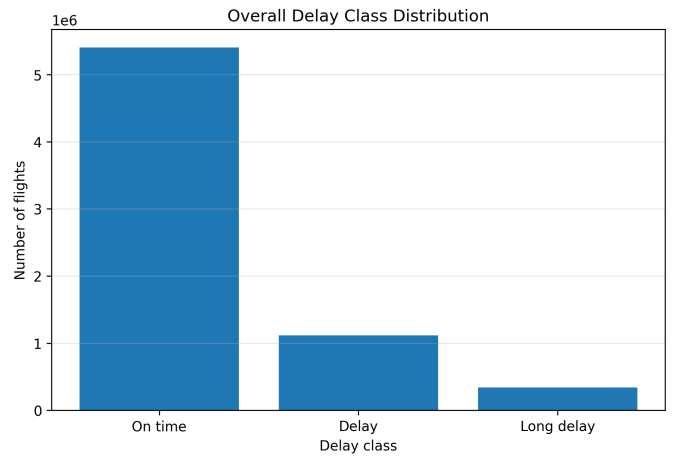


Fig. 1. Overall distribution of the final three-class delay severity target.

B. Target Variable Definition

The target variable was created from arrival delay minutes. Let d_i be the cleaned arrival delay in minutes for flight i , where early arrivals are set to zero. The target y_i is defined as:

$$y_i = \begin{cases} 0, & d_i \leq 15 \\ 1, & 16 \leq d_i \leq 90 \\ 2, & d_i > 90 \end{cases} \quad (1)$$

This target definition balances operational meaning and statistical stability. The first threshold follows the standard 15-minute delay definition used in aviation reporting [?]. The second threshold separates common manageable delays from longer disruptions.

C. Temporal Train, Validation, and Test Split

A temporal split was used instead of a random split. Months 1–9 were used for training, month 10 was used for validation, and months 11–12 were used for final testing. This split is more realistic because it simulates predicting future flights using previous months.

Random splits can produce inflated results in time-dependent data because similar patterns from the same routes, airlines, and seasons can appear in both training and test sets. A temporal split is stricter and gives a more conservative estimate of deployment performance.

Table III shows the class distribution in each split.

TABLE III
TEMPORAL SPLIT CLASS DISTRIBUTION

Split	On time	Delay	Long delay
Train	4,050,886	823,789	253,953
Validation	483,672	95,899	21,999
Test	867,712	197,309	62,199

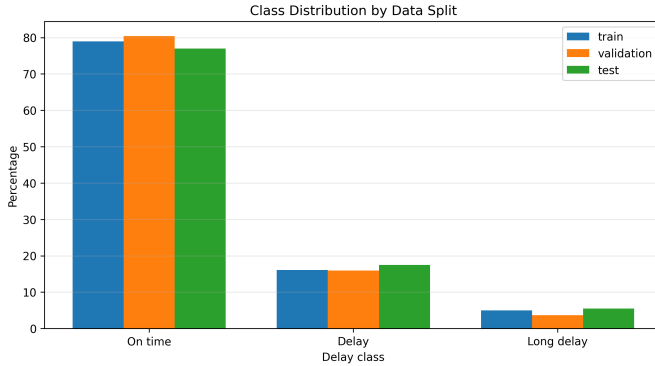


Fig. 2. Class distribution across temporal train, validation, and test splits.

D. Feature Engineering

The feature engineering stage produced two groups of model inputs: categorical features and numerical features.

The categorical features were:

- operating carrier,
- origin airport,
- origin state,
- destination airport,
- destination state,

- route,
- departure time block,
- arrival time block.

The numerical features were:

- distance,
- scheduled departure minutes of day,
- scheduled arrival minutes of day,
- sine and cosine transformations of departure time,
- sine and cosine transformations of arrival time,
- sine and cosine transformations of month,
- sine and cosine transformations of day of week,
- weekend indicator.

Table IV summarizes the feature groups.

TABLE IV
FEATURE GROUPS USED FOR TRAINING

Type	Features
Categorical	Carrier, origin, origin state, destination, destination state, route, departure time block, arrival time block
Numerical	Distance, scheduled departure/arrival minutes, cyclic time features, cyclic month/day features, weekend indicator

Categorical features were encoded as integer IDs and used with embedding layers. Encoders were fitted only on the training set. Unknown validation or test categories were mapped to zero. Numerical features were scaled using a StandardScaler fitted only on the training data.

Time variables were transformed cyclically. For example, scheduled departure time was converted to minutes of day and then encoded as:

$$x_{\sin} = \sin\left(2\pi \frac{m}{1440}\right), \quad (2)$$

$$x_{\cos} = \cos\left(2\pi \frac{m}{1440}\right), \quad (3)$$

where m is the number of minutes since midnight. This representation avoids treating 23:59 and 00:01 as far apart.

E. Categorical Cardinality and Embeddings

High-cardinality categorical variables are difficult to represent with one-hot encoding. The route feature had 6,688 unique categories, while origin and destination each had 350 categories. Embeddings provide a compact dense representation and allow the model to learn similarities between routes, airports, and carriers.

F. Leakage Prevention

A key methodological decision was to avoid data leakage. The model was designed to predict delay severity using only information that would be available before the flight. Therefore, post-flight or delay-cause columns were excluded from the model features.

Excluded columns included actual departure time, departure delay, arrival delay, taxi time, air time, carrier delay, weather

TABLE V
CATEGORICAL FEATURE CARDINALITIES

Feature	Cardinality
ROUTE	6,688
ORIGIN	350
DEST	350
ORIGIN_STATE_ABR	53
DEST_STATE_ABR	53
DEP_TIME_BLK	20
ARR_TIME_BLK	20
OP_UNIQUE_CARRIER	15

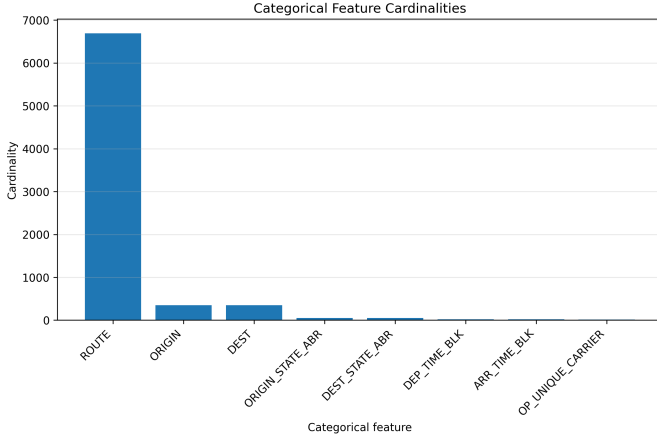


Fig. 3. Cardinality of categorical features used by the embedding-based models.

delay, NAS delay, late aircraft delay, cancellation code, and cleaned arrival delay minutes. These variables would make the model appear much stronger, but they are not valid for a pre-flight prediction setting.

G. Class Weights

Class weights were calculated from the training split to reduce the effect of class imbalance. The raw class weights were:

- On time: 0.4220,
- Delay: 2.0752,
- Long delay: 6.7317.

These values reflect the lower frequency of Delay and Long delay classes. However, very aggressive weighting or oversampling was found to destabilize training in early experiments. The final models used controlled class-weight transformations rather than extreme sampling strategies.

IV. MODEL ARCHITECTURES

A. MLP with Embeddings

The first model was an MLP with categorical embeddings. Each categorical variable was passed through an embedding layer. The embeddings were concatenated with 12 scaled numerical features and passed through a feedforward network.

The hidden layers had sizes 256, 128, and 64. Batch normalization, ReLU activations, and dropout of 0.3 were

applied. The model had 476,856 trainable parameters and served as the baseline neural architecture.

The MLP used AdamW optimization, a learning rate of 0.001, weight decay of 10^{-4} , batch size 4096, and CrossEntropyLoss with softened class weights.

B. Wide and Deep Neural Network

The second model was a Wide and Deep neural network. It had two paths:

- a wide path that mapped numerical features directly to output logits,
- a deep path that processed categorical embeddings and numerical features through dense layers.

The deep path used hidden layers of 384, 192, and 96 units. The final logits were computed as the sum of the wide and deep logits. This architecture was designed to combine simple direct numerical relationships with nonlinear feature interactions.

The model had 559,711 trainable parameters. It used AdamW, learning rate 0.001, weight decay 10^{-4} , and controlled class weights.

C. TabTransformer-Style Model

The third model was a TabTransformer / FT-Transformer style architecture. It represented tabular features as tokens:

- categorical tokens from embedding layers,
- numerical tokens from learned numerical projections,
- one CLS token for global representation.

The token sequence was processed by a Transformer Encoder with:

- $d_{model} = 64$,
- 4 attention heads,
- 3 encoder layers,
- feedforward dimension 256,
- GELU activation,
- dropout 0.15.

The CLS output was concatenated with the numerical features and passed to a classification head with hidden layers of 256 and 128. The model had 689,883 trainable parameters.

Training used AdamW [7], OneCycleLR, mixed precision, gradient clipping, label smoothing, and early stopping. Probability calibration was applied after training using the validation set. This calibration multiplied class probabilities by validation-tuned class multipliers and then renormalized the result. The best multipliers were:

$$[1.0, 0.95, 1.1429].$$

Calibration was not tuned on the test set.

V. EXPERIMENTAL SETUP

All experiments were run on an HP OMEN 16 laptop with an NVIDIA GeForce RTX 4060 Laptop GPU, an Intel Core i9-13900HX CPU, and 32 GB of DDR5 5600 MHz RAM. The batch size was 4096 for all models.

The training setup is summarized in Table VI. Training time is reported because model complexity and training cost are important in practical deep learning workflows.

TABLE VI
TRAINING CONFIGURATION AND TIME

Model	Epochs	Best Epoch	Time (min)
MLP with Embeddings	15	15	24.67
Wide and Deep	15	13	25.07
TabTransformer	7	2	22.89

A. Evaluation Metrics

Accuracy, Weighted F1-score, Macro F1-score, and Balanced Accuracy were considered. Accuracy is useful but can be misleading under imbalance. Weighted F1-score is affected by class support and is therefore also influenced by the majority class. Macro F1-score gives equal weight to each class and was selected as the main metric.

For a class c , precision and recall are:

$$Precision_c = \frac{TP_c}{TP_c + FP_c}, \quad (4)$$

$$Recall_c = \frac{TP_c}{TP_c + FN_c}. \quad (5)$$

The F1-score for class c is:

$$F1_c = 2 \cdot \frac{Precision_c \cdot Recall_c}{Precision_c + Recall_c}. \quad (6)$$

Macro F1 is the average F1 across all classes:

$$MacroF1 = \frac{1}{C} \sum_{c=1}^C F1_c. \quad (7)$$

Because $C = 3$ in this project, each class contributes equally to the final Macro F1-score.

VI. EXPERIMENTAL RESULTS

A. Overall Model Performance

Table VII presents the final test results. The MLP achieved the highest accuracy, but this was mainly because it strongly favored the majority On time class. Wide and Deep reduced accuracy but improved Macro F1. The calibrated TabTransformer obtained the best Macro F1-score and was selected as the final model.

TABLE VII
FINAL TEST PERFORMANCE

Model	Accuracy	Macro F1	Weighted F1
MLP with Embeddings	0.7537	0.3315	0.6856
Wide and Deep	0.6977	0.3765	0.6806
TabTransformer Raw	0.6820	0.3771	0.6738
TabTransformer Calibrated	0.6939	0.3905	0.6801

The results show an important trade-off. The MLP has the highest accuracy but weaker minority-class behavior. The TabTransformer calibrated model has lower accuracy than the MLP, but it provides the most balanced classification behavior according to Macro F1-score.

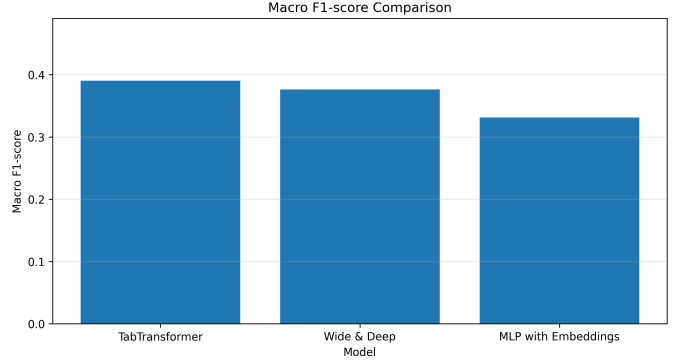


Fig. 4. Macro F1-score comparison of the evaluated models.

B. Per-Class Results

Table VIII shows per-class F1-scores. The On time class is the easiest class for all models. Delay is more difficult, and Long delay is the most difficult class. This is expected because Long delay is the minority class and is likely influenced by external operational conditions not present in the feature set.

TABLE VIII
PER-CLASS F1-SCORE ON THE TEST SET

Model	On time	Delay	Long delay
MLP	0.8604	0.1326	0.0015
Wide and Deep	0.8221	0.2571	0.0503
TabTransformer Raw	0.8099	0.2716	0.0497
TabTransformer Calibrated	0.8213	0.2382	0.1122

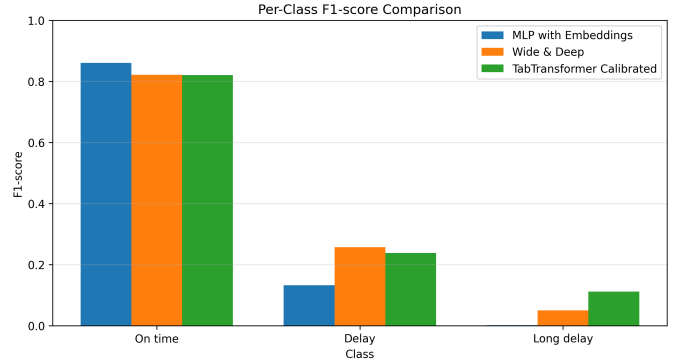


Fig. 5. Per-class F1-score comparison across the main models.

Table IX shows the per-class recall values. The calibrated TabTransformer improves Long delay recall to 0.0921, compared with 0.0007 for the MLP and 0.0309 for Wide and Deep. This is a meaningful improvement because Long delay is the most important minority class.

C. Confusion Matrix Analysis

Table X shows the confusion matrix of the calibrated TabTransformer. Rows represent true labels and columns represent predicted labels. The model correctly classified 732,547 On

TABLE IX
PER-CLASS RECALL ON THE TEST SET

Model	On time	Delay	Long delay
MLP	0.9594	0.0864	0.0007
Wide and Deep	0.8460	0.2555	0.0309
TabTransformer Raw	0.8165	0.2964	0.0297
TabTransformer Calibrated	0.8442	0.2223	0.0921

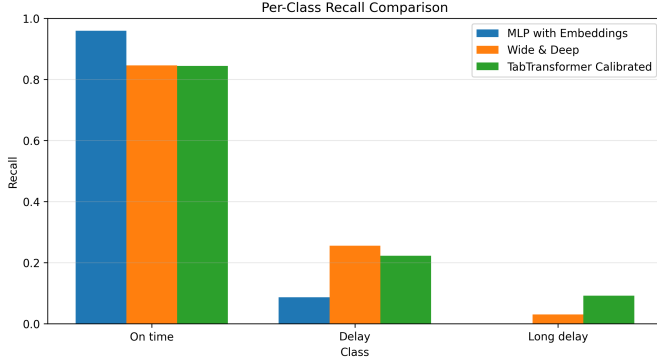


Fig. 6. Per-class recall comparison across the main models.

time flights, 43,871 Delay flights, and 5,728 Long delay flights.

TABLE X
CALIBRATED TABTRANSFORMER CONFUSION MATRIX

True / Predicted	On time	Delay	Long delay
On time	732,547	111,008	24,157
Delay	143,398	43,871	10,040
Long delay	40,307	16,164	5,728

The confusion matrix shows that many Long delay flights are still classified as On time or Delay. This is a limitation of the current feature set. However, compared with the MLP and Wide and Deep models, the calibrated TabTransformer detects substantially more Long delay cases.

D. Training Behavior

The TabTransformer achieved its best validation Macro F1 at epoch 2 and stopped after 7 epochs due to early stopping. This suggests that the model learned useful feature interactions quickly, while later epochs did not improve validation Macro F1. The MLP and Wide and Deep models trained for the full 15 epochs.

VII. GRAPHICAL INFERENCE PROTOTYPE

A small Python graphical application was implemented to demonstrate how the trained model can be used for inference. The application loads the calibrated TabTransformer checkpoint, metadata, categorical encoders, numerical scaler, and route lookup file. The user selects flight information from dropdown lists, including carrier, route, month, day of week, scheduled departure time, and scheduled arrival time.

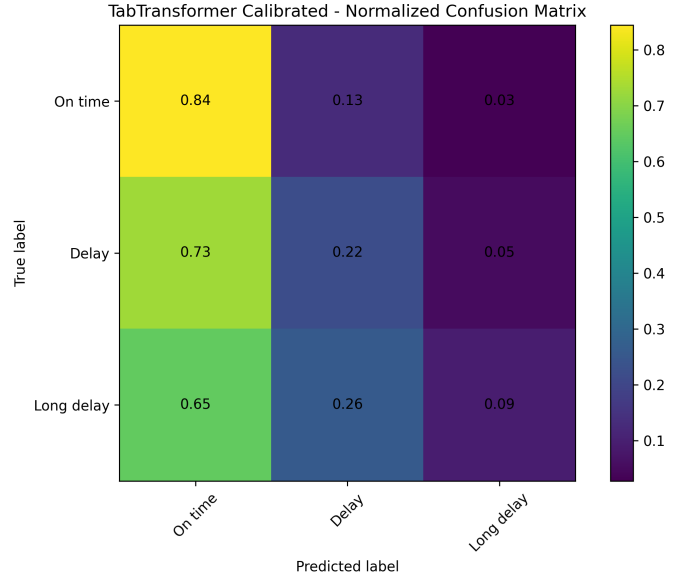


Fig. 7. Normalized confusion matrix for the calibrated TabTransformer.

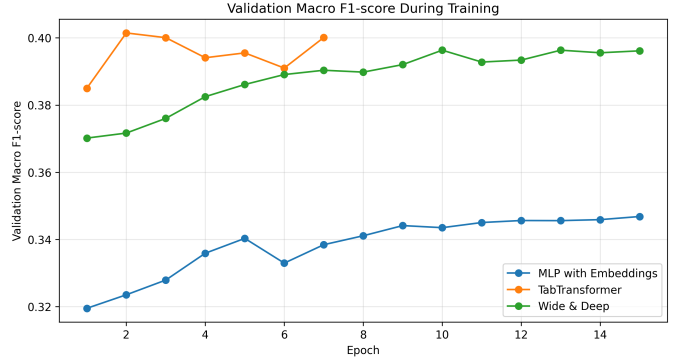


Fig. 8. Validation Macro F1-score during training.

The interface displays calibrated probabilities for the three delay severity classes. This makes the model easier to interpret than only returning a hard class prediction. The prototype is not a production system, but it shows how the trained model can be converted into a practical decision-support tool.

VIII. DISCUSSION

The experimental results should be interpreted according to the difficulty of the task. The model only uses information available before the flight, such as airline, route, origin, destination, schedule, distance, and calendar features. It does not use weather, aircraft rotation, previous flight delay, maintenance events, airport congestion, air traffic control restrictions, or real-time operational disruptions.

These missing factors are likely important for predicting Long delay. A long delay can be caused by unexpected events that are not visible from schedule information alone. Therefore, the relatively low Long delay recall is expected. The model should be interpreted as an early risk estimator rather than a definitive operational predictor.

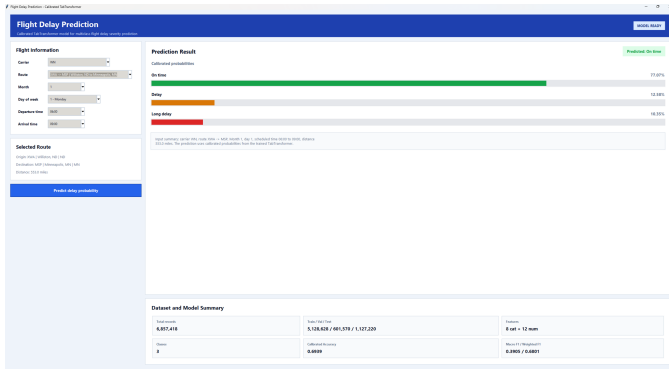


Fig. 9. Graphical inference interface developed in Python to estimate calibrated delay severity probabilities using the trained TabTransformer model.

The temporal split also makes the results more conservative. A random split would likely produce higher scores because similar flight patterns from the same months could appear in both training and testing. In contrast, the temporal split evaluates how well the model generalizes to later months. This better approximates real deployment.

The MLP result illustrates why accuracy is not enough. The MLP achieved the highest accuracy but almost ignored Long delay cases. Wide and Deep reduced accuracy but improved Macro F1 by predicting minority classes more often. The calibrated TabTransformer achieved the best Macro F1 and the best Long delay F1-score. This suggests that attention-based feature interactions and validation-based calibration provide a better balance for this imbalanced multiclass problem.

The improvement over Wide and Deep is moderate, not dramatic. However, it is still meaningful because it occurs in the most difficult part of the problem: minority-class delay severity detection. In real operational settings, missing a Long delay can be more costly than misclassifying an On time flight as a moderate Delay risk. Therefore, selecting the best model by Macro F1 is more appropriate than selecting only by accuracy.

IX. CONCLUSION AND FUTURE WORK

This paper presented a complete deep learning pipeline for temporal flight delay severity prediction using 2025 flight records. The final dataset contained 6,857,418 cleaned model-ready flights. A three-class target was used to classify flights as On time, Delay, or Long delay. This target definition improved stability compared with the original five-class design, which produced severe imbalance.

Three PyTorch models were evaluated: an MLP with embeddings, a Wide and Deep neural network, and a TabTransformer-style architecture. The MLP provided a stable baseline but favored the majority class. Wide and Deep improved balanced performance. The calibrated TabTransformer achieved the best test Macro F1-score of 0.3905 and the best Long delay F1-score among the tested models.

The results are meaningful because the evaluation avoids data leakage and uses a temporal split. However, Long delay

prediction remains difficult because the model does not use external operational features. Future work should integrate weather data, aircraft rotation information, previous flight delay, airport congestion indicators, maintenance events, and real-time air traffic conditions. Additional experiments could also compare deep learning models against gradient boosting baselines, evaluate repeated runs with confidence intervals, and explore cost-sensitive thresholds for operational decision-making.

Finally, the graphical inference prototype could be extended into a web dashboard or operational monitoring tool. This would make the model easier to test with different flight scenarios and could support practical delay risk estimation.

REFERENCES

- [1] Bureau of Transportation Statistics, U.S. Department of Transportation, "Airline On-Time Performance Data," TranStats Database Profile. [Online]. Available: https://transtats.bts.gov/DatabaseInfo.asp?QO_VQ=EFD&DB_URL=. Accessed: Apr 2026.
- [2] J. Huo, H. Zhang, and V. A. Profillidis, "The prediction of flight delay: A big data-driven machine learning approach," in *Proc. International Conference on Transportation and Development*, 2020.
- [3] H.-T. Cheng, L. Koc, J. Harmsen, T. Shaked, T. Chandra, H. Aradhye, G. Anderson, G. Corrado, W. Chai, M. Ispir, R. Anil, Z. Haque, L. Hong, V. Jain, X. Liu, and H. Shah, "Wide & Deep learning for recommender systems," in *Proc. 1st Workshop on Deep Learning for Recommender Systems*, 2016, pp. 7–10.
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [5] X. Huang, A. Khetan, M. Cvitkovic, and Z. Karnin, "TabTransformer: Tabular data modeling using contextual embeddings," arXiv preprint arXiv:2012.06678, 2020.
- [6] Y. Gorishniy, I. Rubachev, V. Khurlov, and A. Babenko, "Revisiting deep learning models for tabular data," in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 18932–18943.
- [7] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2019.
- [8] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. 34th International Conference on Machine Learning*, 2017, pp. 1321–1330.
- [9] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.